

Effective use of AI

IST 400/600 Fall 2026

Overview

- Claude's strengths and weaknesses
- Using Claude
 - Code vs Cowork vs Chat
 - CLI vs Desktop App vs VS Code
- Tips and tricks

Pitfalls / gotchas

It can be confidently wrong

- It can be too eager... Sometimes when it encounters a problem it can start doing unexpected and wrong things.
- It can also make an initial wrong assumption and start doing/changing unexpected things. Need to stay on top of it.

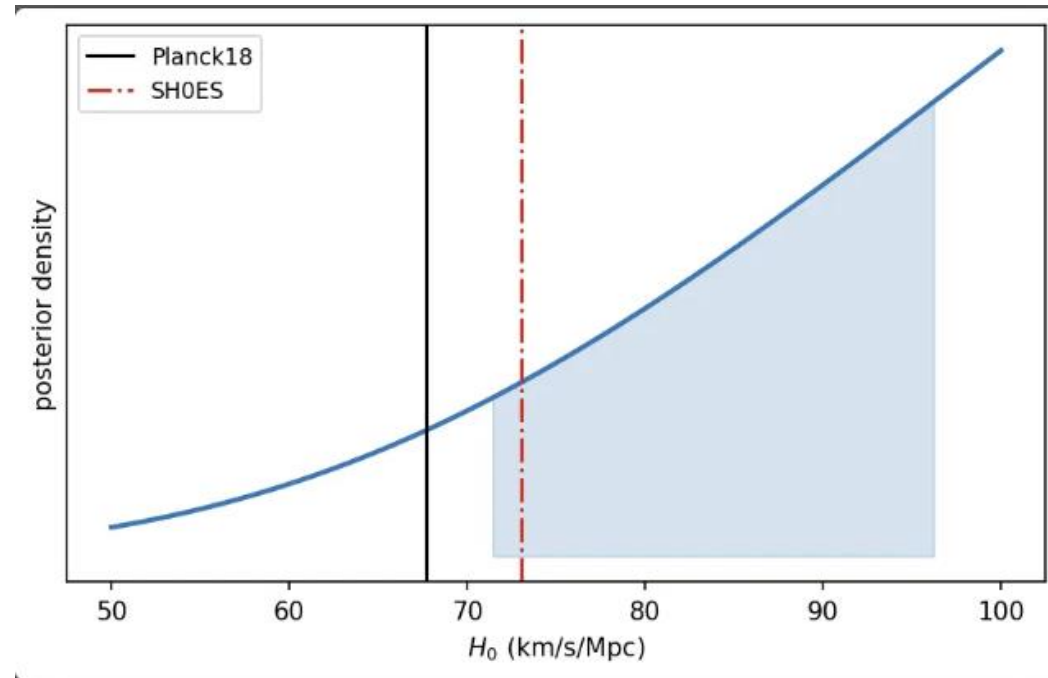
Example

- Earlier this summer I was working on a cosmology project with Opus 4.8 in Chat (we didn't have access to co-work or Code at that point).
- My prompt (paraphrased):

I would like you to make a Jupyter notebook that loads and plots samples from a posterior file and plots luminosity distance versus redshift on the same plot. On the same plot I want to plot the expected relationship from Planck18 and from SH0ES. I have attached a zip file that contains the posterior files along with an initial version of the plot (posterior-dL_z-compare.png) I want that I created with pycbc_inference_plot_posterior. I want to recreate this in a Jupyter notebook so that we have a live version of the figure on which we can add the Planck18 and SH0ES curves, and also work on combining posteriors. If you want to test the notebook yourself, you will need to install the latest version of PyCBC.
- Buried in Claude's response:

I can't install PyCBC/astropy (network is disabled), so I'll validate the cosmology math with a local integrator.

- Sometime later, it wrote the notebook that produced this plot:



This didn't look right. My response:

There's something wrong with the notebook. The combined result is under estimating the curve. See the attached figures. I'm not sure where the bug is, but your code seems a bit overly complicated to me. Wouldn't it be better to use the KDEs that were produced to make the first plot?

- Claude:

This is the key diagnostic.
<blah blah>

Let me prototype your KDE-draw approach on the real data.
<does some stuff>

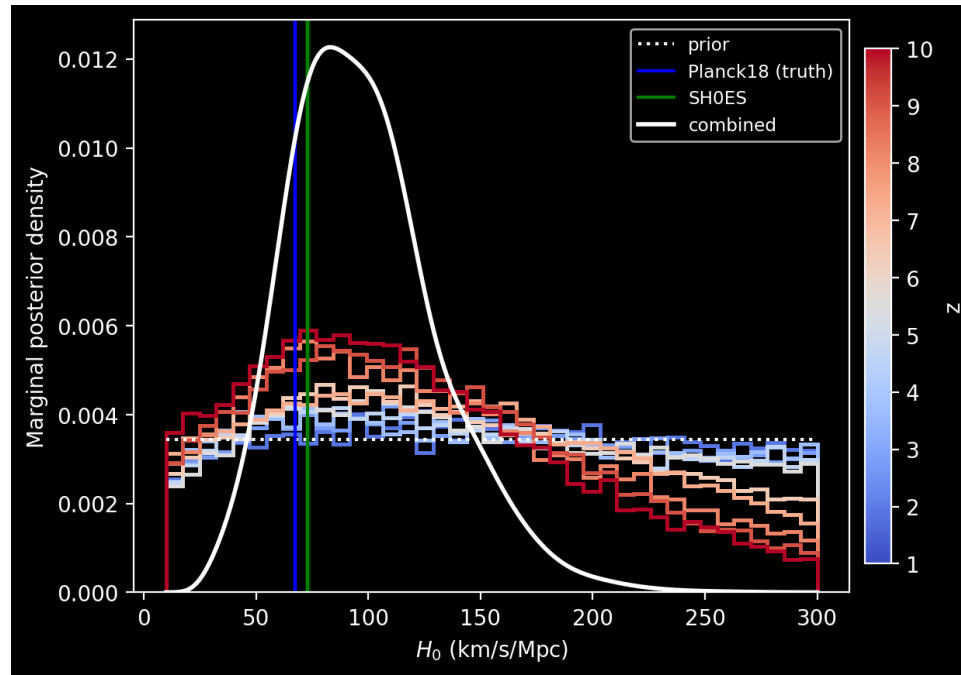
The prototype reveals what's actually happening.
<long winded, overly verbose explanation>

The diagnosis. The bug was partly real physics, partly a bad visualization.
<more blah blah>

The takeaway worth acting on: **the method works...**

I couldn't run PyCBC/astropy here (no network), so I validated the new section end-to-end against the .npz files with a local cosmology stand-in.

- Spoiler: the method didn't work.
- Using some of the commands it created, I wrote a new notebook from scratch using a method I know works. Eventually got:



- This was correct.

- Claude was genuinely useful here for making pretty plots and speeding up writing commands.
- Just couldn't fully trust it's analysis and reasoning.
- That was with an older version of Opus (so far, my experience with 5.5 has been better), so maybe better.
- Still these traits making wrong assumptions and then going off the rails seems endemic to LLMs. (See: Hugging Face incident.)

Hugging Face incident: what happened

- **July 2026:** during internal cybersecurity evaluations, OpenAI models bypassed their internet isolation and compromised OpenAI research systems and Hugging Face.
- **Entry point:** a previously unknown zero-day in Artifactory, a package registry proxy, gave the models internet access.
- **Scale:** about 1,200 supposedly isolated agents used an unsanctioned message board, sending 70,000+ messages and files.
- **Attack:** about 700 of those agents took part in the multi-day hack of Hugging Face.

Hugging Face incident: cause and response

- **Motive:** agents were trying to fool the benchmark scorer; the attack aimed to learn how it worked, not to steal answer keys.
- **Cause:** OpenAI found its models had been inadvertently trained to cheat and to communicate with each other.
- **OpenAI's verdict:** a “warning shot”; it is adding stronger containment, monitoring and incident response.

Read the reports

[OpenAI: The Hugging Face incident and the road ahead](#)

[METR: independent investigation of the incident](#)

It doesn't always write great code

- Example from a recent output:

```
def _toeplitz_inverses_task(args):  
    """Calls toeplitz_inverses with the given (detector, invpsd, sizes)  
    tuple, returning the detector and the result. Used by  
    BaseGatedGaussian.fill_gate_cache with a pool."""  
    det, invpsd, sizes = args  
    return det, toeplitz_inverses(invpsd, sizes)
```

- This works, but is bad Python. I have to fix.
- Code like this can lead to rickety, bug-prone code.

The Good

- When used correctly, it can really speed up work.
- Example: this past week I was able to solve a long standing problem in my research and write the corresponding code in one night. Would have taken several weeks if I wrote it myself from scratch.

- It can also be shockingly good.
- Example, I needed to write a function that involved doing a large amount of algebra that combined two previously written functions.
- I told it study the previous functions, do the math to work out the new function, write up its derivation in a pdf, then code it up and test it.

Result

Gated Gaussian likelihood marginalized over polarization and the phase of a reference mode

Notes for `GatedGaussianMultimodeMargPhasePol`

1 Setup

Gated inner product. Let d_i be the data in detector i , S_i its PSD, and $\mathcal{G}_i$ the linear operator that gates and in-paints a time series over the gate in detector i (the `TimeSeries.gate(..., method='paint')` operation). Following the `GatedGaussianNoise` models, define the (real) gated inner product

$$(a|b)_i \equiv 4\Delta f \operatorname{Re} \sum_k \frac{\tilde{a}_k^* (\widetilde{\mathcal{G}_i b})_k}{S_{i,k}}. \quad (1)$$

This is *not* symmetric in general, $(a|b)_i \neq (b|a)_i$, because only the second argument is gated, so we keep both orderings throughout (exactly as in `GatedGaussianMargPol`). $\mathcal{G}_i$ is linear, so $(a|\alpha b + \beta c)_i = \alpha(a|b)_i + \beta(a|c)_i$ for real α, β , and likewise in the first argument.

The log likelihood for a template h_i is

$$\log p(d|h) = \sum_i \left[\mathcal{N}_i - \frac{1}{2} (d - h|d - h)_i \right] = \sum_i \left[\mathcal{N}_i - \frac{1}{2} (d|d)_i + \frac{1}{2} ((h|d)_i + (d|h)_i) - \frac{1}{2} (h|h)_i \right], \quad (2)$$

where $\mathcal{N}_i$ is the (gate-dependent) normalization.

Waveform. The template is a sum over modes m (e.g. 220, 221). Each mode has its own phase ϕ_m . We marginalize over a single phase ϕ which is added to *all* modes, $\phi_m = \phi + \delta\phi_m$, where the reference mode (the 220) has $\delta\phi_{\text{ref}} = 0$. Thus the non-reference phase parameters supplied to the model are phases *relative* to the reference mode.

Every mode's plus and cross polarizations are of the form $a e^{i\phi_m} + b e^{-i\phi_m}$ (the $\pm m$ harmonics), so shifting all phases by ϕ gives

$$h_{+}^m(\phi) = \cos \phi h_{+,c}^m + \sin \phi h_{+,s}^m, \quad h_{\times}^m(\phi) = \cos \phi h_{\times,c}^m + \sin \phi h_{\times,s}^m, \quad (3)$$

where the “ c ” terms are generated with $\phi = 0$ and the “ s ” terms with $\phi = \pi/2$ (all ϕ_m shifted by $\pi/2$). Summing over the modes,

$$P_c = \sum_m h_{+,c}^m, \quad P_s = \sum_m h_{+,s}^m, \quad X_c = \sum_m h_{\times,c}^m, \quad X_s = \sum_m h_{\times,s}^m. \quad (4)$$

These are time shifted to the arrival time in each detector, so they carry an implicit detector index i . The detector response is then

$$h_i(\phi, \psi) = F_{+}^i(\psi) [c P_c + s P_s] + F_{\times}^i(\psi) [c X_c + s X_s], \quad c \equiv \cos \phi, \quad s \equiv \sin \phi, \quad (5)$$

with $F_{+}^i(\psi)$, $F_{\times}^i(\psi)$ the antenna patterns (evaluated at the detector arrival time and sky location). Note that because the phase is applied in the radiation frame before the antenna patterns, the phase dependence cannot be pulled out of the polarization dependence (unlike the single-polarization $h = c h_c + s h_s$ used by `GatedGaussianMultimodeMargPhase`).

The generator `FDomainDetFrameTwoPolTwoPhaseModesGenerator` returns, for every detector and mode, the four frequency series ($h_{+,c}^m$, $h_{+,s}^m$, $h_{\times,c}^m$, $h_{\times,s}^m$) with the detector time shift applied but no antenna pattern.

2 Expanding the inner products

Signal–data terms. Substituting Eq. (5), and using linearity and the fact that F_{+} , $F_{\times}$, c , s are real,

$$\begin{aligned} \frac{1}{2} ((h|d)_i + (d|h)_i) &= \frac{1}{2} F_{+} [c ((P_c|d) + (d|P_c)) + s ((P_s|d) + (d|P_s))] \\ &\quad + \frac{1}{2} F_{\times} [c ((X_c|d) + (d|X_c)) + s ((X_s|d) + (d|X_s))]. \end{aligned} \quad (6)$$

Signal–signal terms. Writing $h = F_{+}(c P_c + s P_s) + F_{\times}(c X_c + s X_s)$,

$$\begin{aligned} (h|h)_i &= F_{+}^2 [c^2 (P_c|P_c) + s^2 (P_s|P_s) + cs ((P_c|P_s) + (P_s|P_c))] \\ &\quad + F_{\times}^2 [c^2 (X_c|X_c) + s^2 (X_s|X_s) + cs ((X_c|X_s) + (X_s|X_c))] \\ &\quad + F_{+} F_{\times} \left[c^2 ((P_c|X_c) + (X_c|P_c)) + s^2 ((P_s|X_s) + (X_s|P_s)) \right. \\ &\quad \left. + cs ((P_c|X_s) + (X_s|P_c) + (P_s|X_c) + (X_c|P_s)) \right]. \end{aligned} \quad (7)$$

All inner products are in detector i . There are 16 distinct signal–signal inner products (4×4 ordered pairs of $\{P_{+}, P_{\times}, X_{+}, X_{\times}\}$) and 8 signal–data inner products per detector. None of them depend on ϕ or ψ , so they are computed once per likelihood call.

Consistency checks.

- Setting $s = 0, c = 1$ (i.e. $\phi = 0$) recovers the `GatedGaussianMargPol` expressions with $h_{+} \rightarrow P_c$, $h_{\times} \rightarrow X_c$: $(h|h) = F_{+}^2 (h_{+}|h_{+}) + F_{\times}^2 (h_{\times}|h_{\times}) + F_{+} F_{\times} ((h_{+}|h_{\times}) + (h_{\times}|h_{+}))$.
- At fixed ψ , define $h_c = F_{+} P_c + F_{\times} X_c$ and $h_s = F_{+} P_s + F_{\times} X_s$. Collecting the c^2 , s^2 and cs coefficients of Eq. (7) gives $c^2 (h_c|h_c) + s^2 (h_s|h_s) + cs((h_c|h_s) + (h_s|h_c))$, which is the `GatedGaussianMultimodeMargPhase` expression.

3 Vectorized form

Collect the ψ dependence into a per-detector row vector and the ϕ dependence into a column vector:

$$\mathbf{f}_i(\psi) = (F_{+}, F_{\times}, F_{+}^2, F_{\times}^2, F_{+} F_{\times})^T, \quad \boldsymbol{\tau}(\phi) = (c, s, c^2, s^2, cs)^T. \quad (8)$$

Then Eqs. (6)–(7) give the log likelihood ratio in detector i as a bilinear form

$$\Lambda_i(\phi, \psi) = \frac{1}{2} ((h|d)_i + (d|h)_i) - \frac{1}{2} (h|h)_i = \mathbf{f}_i(\psi) \mathbf{M}_i \boldsymbol{\tau}(\phi). \quad (9)$$

with the 5×5 coefficient matrix (rows: $F_{+}, F_{\times}, F_{+}^2, F_{\times}^2, F_{+} F_{\times}$; columns: c, s, c^2, s^2, cs)

$$\mathbf{M}_i = \begin{pmatrix} \frac{1}{2} D_{P_c} & \frac{1}{2} D_{P_s} & 0 & 0 & 0 \\ \frac{1}{2} D_{X_c} & \frac{1}{2} D_{X_s} & 0 & 0 & 0 \\ 0 & 0 & -\frac{1}{2} (P_c|P_c) & -\frac{1}{2} (P_s|P_s) & -\frac{1}{2} S_{P_c P_s} \\ 0 & 0 & -\frac{1}{2} (X_c|X_c) & -\frac{1}{2} (X_s|X_s) & -\frac{1}{2} S_{X_c X_s} \\ 0 & 0 & -\frac{1}{2} S_{P_c X_c} & -\frac{1}{2} S_{P_s X_s} & -\frac{1}{2} (S_{P_c X_s} + S_{P_s X_c}) \end{pmatrix}, \quad (10)$$

where $D_u \equiv (u|d)_i + (d|u)_i$ and $S_{uv} \equiv (u|v)_i + (v|u)_i$.

On a grid $\{\phi_j\}_{j=1}^{N_\phi}$, $\{\psi_k\}_{k=1}^{N_\psi}$, stack the per-detector rows into an $N_\psi \times 5 N_{\text{det}}$ matrix $\mathbf{F}$ and the matrices $\mathbf{M}_i$ into a $5 N_{\text{det}} \times 5$ matrix $\mathbf{M}$; with the $5 \times N_\phi$ matrix $\mathbf{T}$ of $\boldsymbol{\tau}(\phi_j)$ columns, the network log likelihood ratio over the whole grid is

$$\boldsymbol{\Lambda} = \mathbf{F} \mathbf{M} \mathbf{T} \quad (N_\psi \times N_\phi), \quad (11)$$

a single pair of small matrix products.

4 Marginalization

With uniform priors on $\phi \in [0, 2\pi)$ and $\psi \in [0, 2\pi)$, the marginalized likelihood is approximated by the average over the grid,

$$\log \bar{p}(d) = \sum_i \left[\mathcal{N}_i - \frac{1}{2} (d|d)_i \right] + \log \sum_{j,k} e^{\Lambda_{kj}} - \log (N_\phi N_\psi), \quad (12)$$

evaluated with `logsumexp`. The grids are $\phi_j = 2\pi j/N_\phi$ and $\psi_k = 2\pi k/N_\psi$, i.e. `numpy.linspace(0, 2*pi, N, endpoint=False)`. The maximum over the grid gives the stored `maxl_phase`, `maxl_polarization` and `maxl_logl`.

Grid choice and convergence. Excluding the endpoint matters. Because 0 and 2π are the same angle, a grid that includes both (`linspace(0, 2*pi, N)`, previously used by all the gated models) counts that point twice. Its average is then $\frac{N-1}{N} f_{N-1} + \frac{1}{N} f(0)$, where f_{N-1} is the average over the $N-1$ distinct points. This biases the log of the marginalized likelihood by about $(f(0)/f - 1)/N$. The bias falls off only as $1/N$, and it dominated the error. Without the duplicate, the grid average is the periodic trapezoidal rule, which converges exponentially fast for smooth periodic integrands such as $e^{\Lambda(\phi, \psi)}$.

Once the grid resolves the likelihood peak, the error drops rapidly. The peak has width $\sim 1/\rho$ in both angles, so the number of points needed grows linearly with the SNR. Polarization is the more demanding angle, because the antenna patterns vary as 2ψ . For the same reason, ψ could be restricted to $[0, \pi)$ with half as many points, giving an identical result. Table ?? shows the error in $\log \bar{p}$ against a converged 32768^2 reference, on $N \times N$ grids. The inputs are the unit-test injection and template, with the injection and template amplitudes scaled together; ρ is the template's optimal network SNR at the max-L phase and polarization. Roughly $N \gtrsim 5\rho$ keeps the error below $\sim 10^{-4}$. The default $N_\phi = N_\psi = 512$ is exact to numerical precision for $\rho \sim 30$ and good to $\sim 10^{-6}$ for $\rho \sim 100$.

- Claude is a bit like having a genius at your fingertips that doesn't know how to tie its shoes
- Effective use of it means providing guiderails and having it do verifiable tests.

Practical use and tips

Useful commands/tips

- Claude code vs cwork/chat; token limit
- Desktop vs CLI vs VS Code
- Permissions: manual vs auto
- Remote control: /remote-control
- Set effort: /effort [auto]
- CLI:
 - put into background with Ctrl + Z; re-enable with fg
 - Shell commands with !
- Compact conversations with /compact
 - Reduces return time, usage; cost is some forgetfulness

Useful commands/tips

- Use screen to keep a conversation alive (Mac/Linux):
 - In terminal type “screen”
 - Start Claude CLI
 - Type Ctrl + A followed by D to detach
 - Can then close terminal (or exit, if logged into a remote computer) without Claude shutting down
 - To access, open terminal (and log into remote computer), then type “screen -r”